In [52]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")
```

In [53]:
```
df=pd.read_csv("restaurant_customer_satisfaction.csv")
```

In [54]:
```
df
```

Out[54]:

	CustomerID	Age	Gender	Income	VisitFrequency	AverageSpend	PreferredCuisine	TimeOfVisit	GroupSize	DiningOccasion
0	718	33	Male	117671.62910	Monthly	179.798439	Mexican	Breakfast	8	Celebration
1	792	57	Female	105429.58610	Weekly	176.195249	Mexican	Lunch	3	Celebration
2	955	56	Male	125265.09480	Rarely	178.713392	American	Breakfast	9	Celebration
3	965	58	Female	133880.88430	Monthly	176.967930	Italian	Lunch	1	Business
4	1056	32	Female	115413.21850	Weekly	194.071246	Chinese	Dinner	7	Business
...	...	...	...	...	...	...	...	...	...	...
1495	1974	59	Male	47143.24962	Rarely	68.631166	Italian	Dinner	5	Casual
1496	2013	22	Female	47800.79960	Weekly	62.451076	American	Dinner	8	Celebration
1497	2041	59	Female	74776.55598	Weekly	124.713378	Mexican	Lunch	3	Casual
1498	2086	31	Male	68684.36744	Weekly	130.267592	Chinese	Dinner	2	Celebration
1499	2115	62	Female	14961.50882	Rarely	35.224103	American	Lunch	1	Business

1500 rows × 19 columns

In [55]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1500 entries, 0 to 1499
Data columns (total 19 columns):
 #   Column                Non-Null Count  Dtype  
---  ------                --------------  -----  
 0   CustomerID            1500 non-null   int64  
 1   Age                   1500 non-null   int64  
 2   Gender                1500 non-null   object 
 3   Income                1500 non-null   float64
 4   VisitFrequency        1500 non-null   object 
 5   AverageSpend          1500 non-null   float64
 6   PreferredCuisine      1500 non-null   object 
 7   TimeOfVisit           1500 non-null   object 
 8   GroupSize             1500 non-null   int64  
 9   DiningOccasion        1500 non-null   object 
 10  MealType              1500 non-null   object 
 11  OnlineReservation     1500 non-null   int64  
 12  DeliveryOrder         1500 non-null   int64  
 13  LoyaltyProgramMember  1500 non-null   int64  
 14  WaitTime              1500 non-null   float64
 15  ServiceRating         1500 non-null   int64  
 16  FoodRating            1500 non-null   int64  
 17  AmbianceRating        1500 non-null   int64  
 18  HighSatisfaction      1500 non-null   int64  
dtypes: float64(3), int64(10), object(6)
memory usage: 222.8+ KB
```

In [56]:
```
df.head()
```

Out[56]:

	CustomerID	Age	Gender	Income	VisitFrequency	AverageSpend	PreferredCuisine	TimeOfVisit	GroupSize	DiningOccasion	Mea
0	718	33	Male	117671.6291	Monthly	179.798439	Mexican	Breakfast	8	Celebration	Tak
1	792	57	Female	105429.5861	Weekly	176.195249	Mexican	Lunch	3	Celebration	Tak
2	955	56	Male	125265.0948	Rarely	178.713392	American	Breakfast	9	Celebration	Tak
3	965	58	Female	133880.8843	Monthly	176.967930	Italian	Lunch	1	Business	D
4	1056	32	Female	115413.2185	Weekly	194.071246	Chinese	Dinner	7	Business	D

In [57]:
```
df.shape
```

Out[57]:
```
(1500, 19)
```

In [58]:
```
for col in df:
    print(col,":",df[col].unique())
    print("************************************")
```

```
CustomerID : [ 718  792  955 ... 2041 2086 2115]
**************************************
Age : [33 57 56 58 32 41 69 34 46 38 39 51 49 54 66 67 28 42 36 35 21 19 62 24
 30 26 43 61 25 20 18 31 29 50 59 22 27 68 65 53 64 40 48 55 52 44 60 47
 63 23 45 37]
**************************************
Gender : ['Male' 'Female']
**************************************
Income : [117671.6291  105429.5861  125265.0948  ...  74776.55598  68684.36744
  14961.50882]
**************************************
VisitFrequency : ['Monthly' 'Weekly' 'Rarely' 'Daily']
**************************************
AverageSpend : [179.798439   176.1952492  178.7133919  ... 124.7133778  130.2675917
  35.22410277]
**************************************
PreferredCuisine : ['Mexican' 'American' 'Italian' 'Chinese' 'Indian']
**************************************
TimeOfVisit : ['Breakfast' 'Lunch' 'Dinner']
**************************************
GroupSize : [8 3 9 1 7 2 4 5 6]
**************************************
DiningOccasion : ['Celebration' 'Business' 'Casual']
**************************************
MealType : ['Takeaway' 'Dine-in']
**************************************
OnlineReservation : [1 0]
**************************************
DeliveryOrder : [1 0]
**************************************
LoyaltyProgramMember : [1 0]
**************************************
WaitTime : [39.46 25.25 44.84 ...  4.39 20.62 24.66]
**************************************
ServiceRating : [1 2 3 4 5]
**************************************
FoodRating : [1 2 3 4 5]
**************************************
AmbianceRating : [1 2 3 4 5]
**************************************
```

```
HighSatisfaction : [0 1]
************************************
```

In [59]:
```
#checking for null values
df.isnull().sum()
```

Out[59]:
```
CustomerID             0
Age                    0
Gender                 0
Income                 0
VisitFrequency         0
AverageSpend           0
PreferredCuisine       0
TimeOfVisit            0
GroupSize              0
DiningOccasion         0
MealType               0
OnlineReservation      0
DeliveryOrder          0
LoyaltyProgramMember   0
WaitTime               0
ServiceRating          0
FoodRating             0
AmbianceRating         0
HighSatisfaction       0
dtype: int64
```

In [60]:
```
df.describe()
```

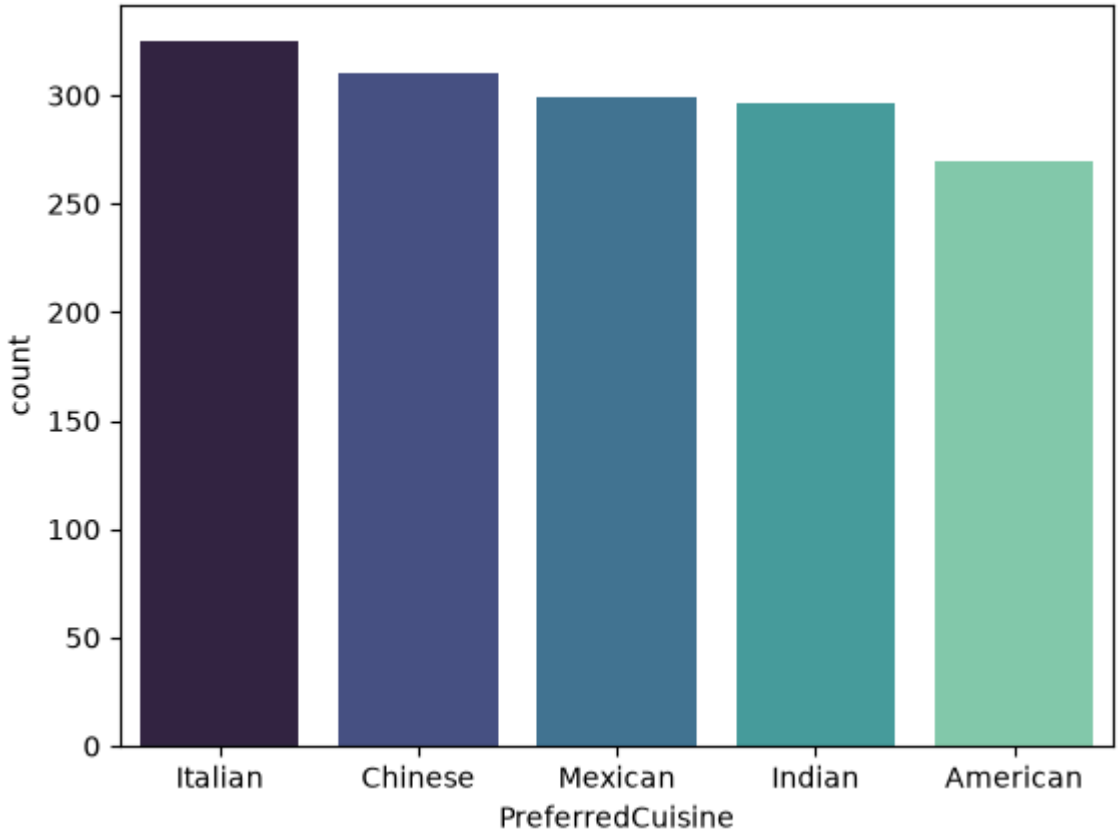
Out[60]:

	CustomerID	Age	Income	AverageSpend	GroupSize	OnlineReservation	DeliveryOrder	LoyaltyProgramMember	1
count	1500.000000	1500.000000	1500.000000	1500.000000	1500.000000	1500.000000	1500.000000	1500.000000	
mean	1403.500000	43.832000	86167.137737	131.118761	5.035333	0.296667	0.405333	0.480000	
std	433.157015	14.967157	38639.661987	42.983147	2.558864	0.456941	0.491120	0.499766	
min	654.000000	18.000000	10099.789500	18.968962	1.000000	0.000000	0.000000	0.000000	
25%	1028.750000	31.750000	51641.794042	104.709636	3.000000	0.000000	0.000000	0.000000	
50%	1403.500000	44.000000	86166.682450	142.851472	5.000000	0.000000	0.000000	0.000000	
75%	1778.250000	57.000000	119020.972075	162.277274	7.000000	1.000000	1.000000	1.000000	
max	2153.000000	69.000000	157945.271200	214.211907	9.000000	1.000000	1.000000	1.000000	

In [61]:
```
df["PreferredCuisine"].value_counts()
```

Out[61]:
```
PreferredCuisine
Italian     325
Chinese     310
Mexican     299
Indian      296
American    270
Name: count, dtype: int64
```

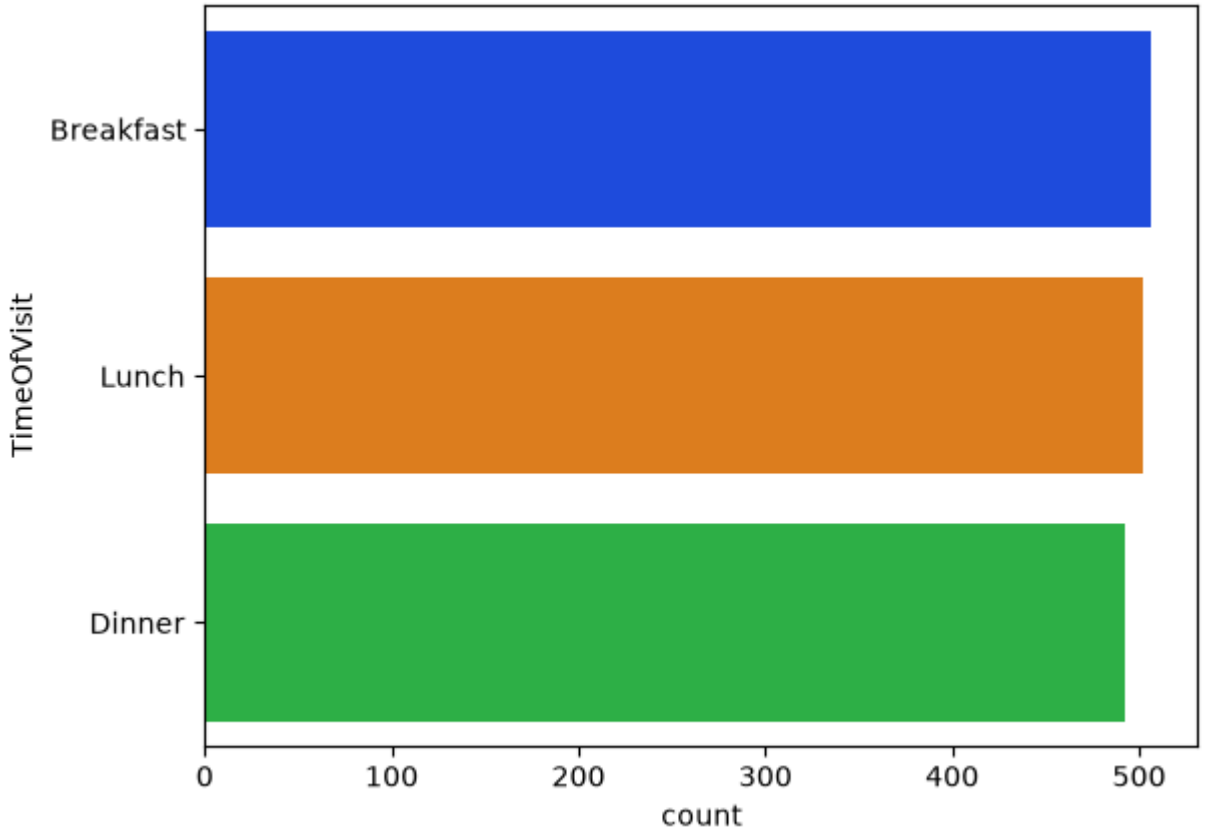
In [62]:
```
sns.countplot(data=df,x="PreferredCuisine",palette="mako",order=df["PreferredCuisine"].value_counts().index)
plt.show()
```



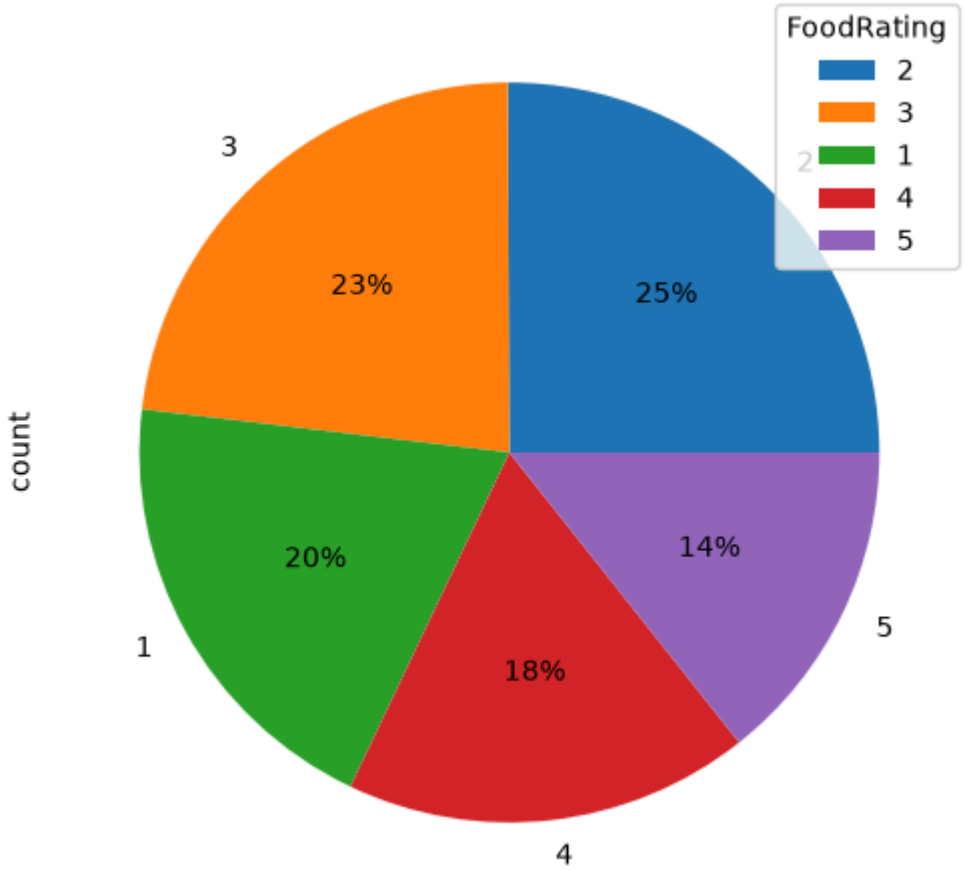
In [63]:
```
df['TimeOfVisit'].value_counts()
```

Out[63]:
```
TimeOfVisit
Breakfast    506
Lunch        502
Dinner       492
Name: count, dtype: int64
```

In [64]:
```
sns.countplot(data=df,y="TimeOfVisit",palette="bright",order=df["TimeOfVisit"].value_counts().index)
plt.show()
```



In [65]:
```
plt.figure(figsize=(10,6))
df["FoodRating"].value_counts().plot(kind="pie",autopct="%.f%%")  #%.2f%%== 2 decimals
plt.legend(title="FoodRating",loc="upper right")
plt.show()
```

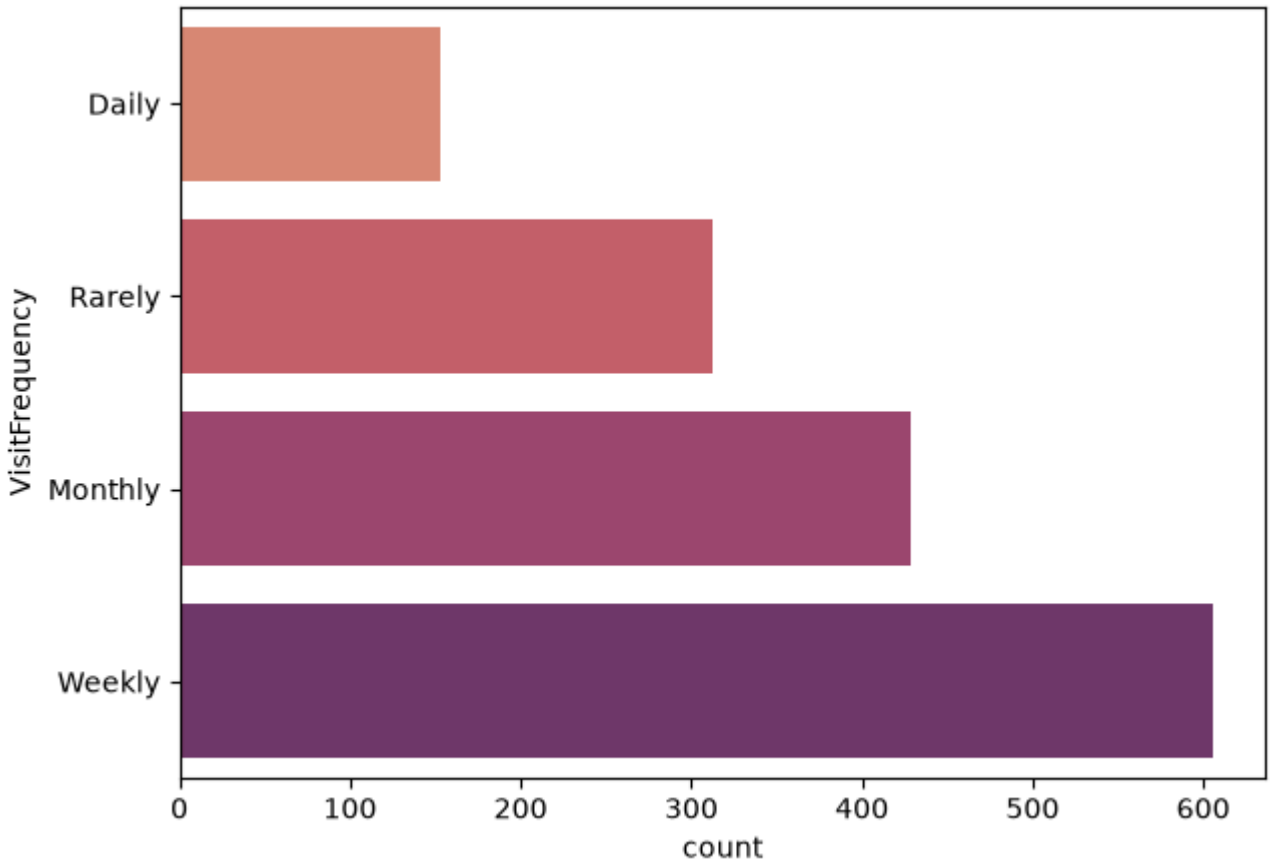


In [66]:
```
df["VisitFrequency"].value_counts()#3-Weekly 2-Rarely 1-Monthly 0-Daily
```

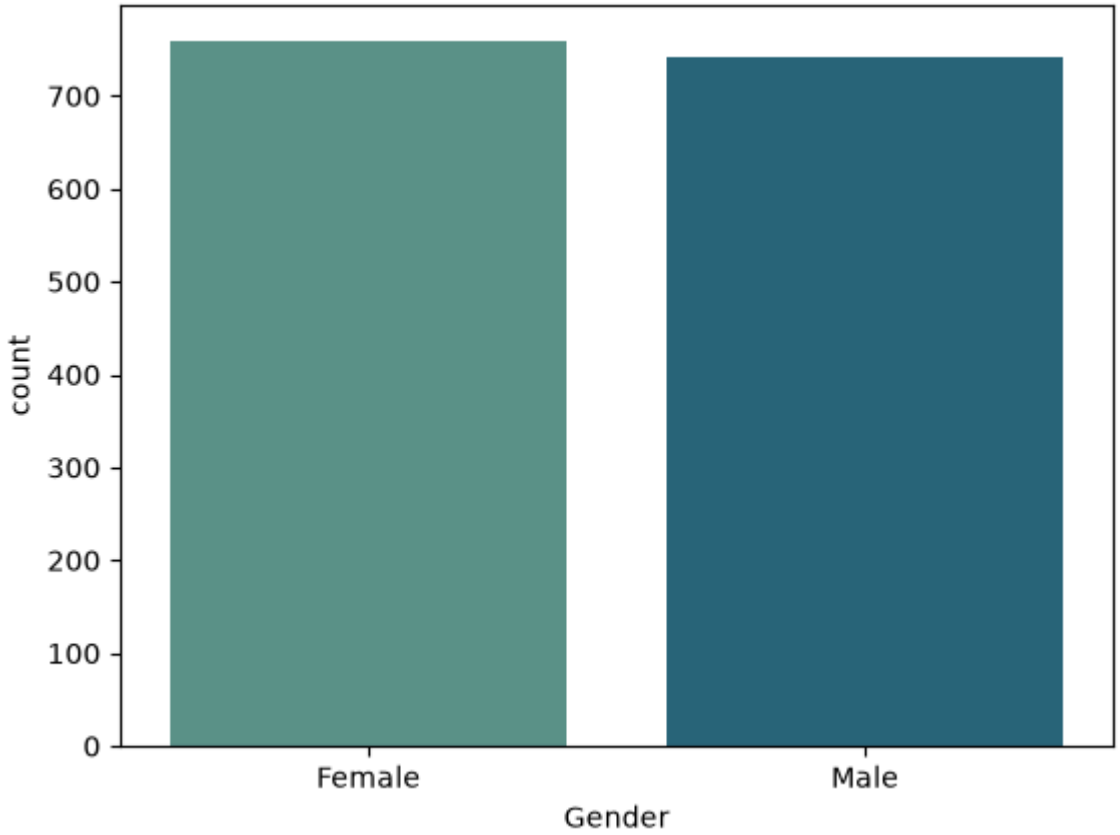
Out[66]:
```
VisitFrequency
Weekly     606
Monthly    428
Rarely     313
Daily      153
Name: count, dtype: int64
```

In [67]:
```
plt.figure(figsize=(7,5))
sns.countplot(data=df,y="VisitFrequency",palette="flare",order=df["VisitFrequency"].value_counts(ascending=True).index)
```

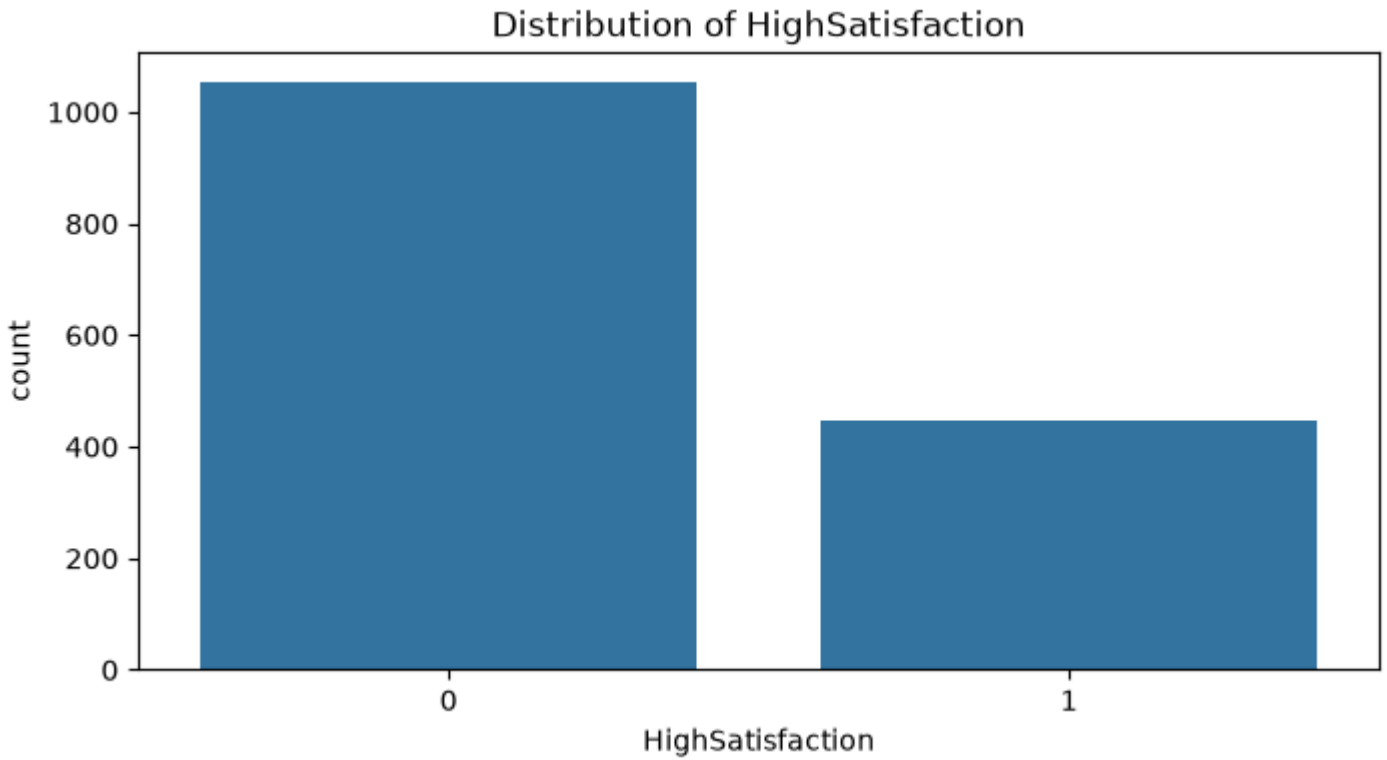
```
plt.show()
```



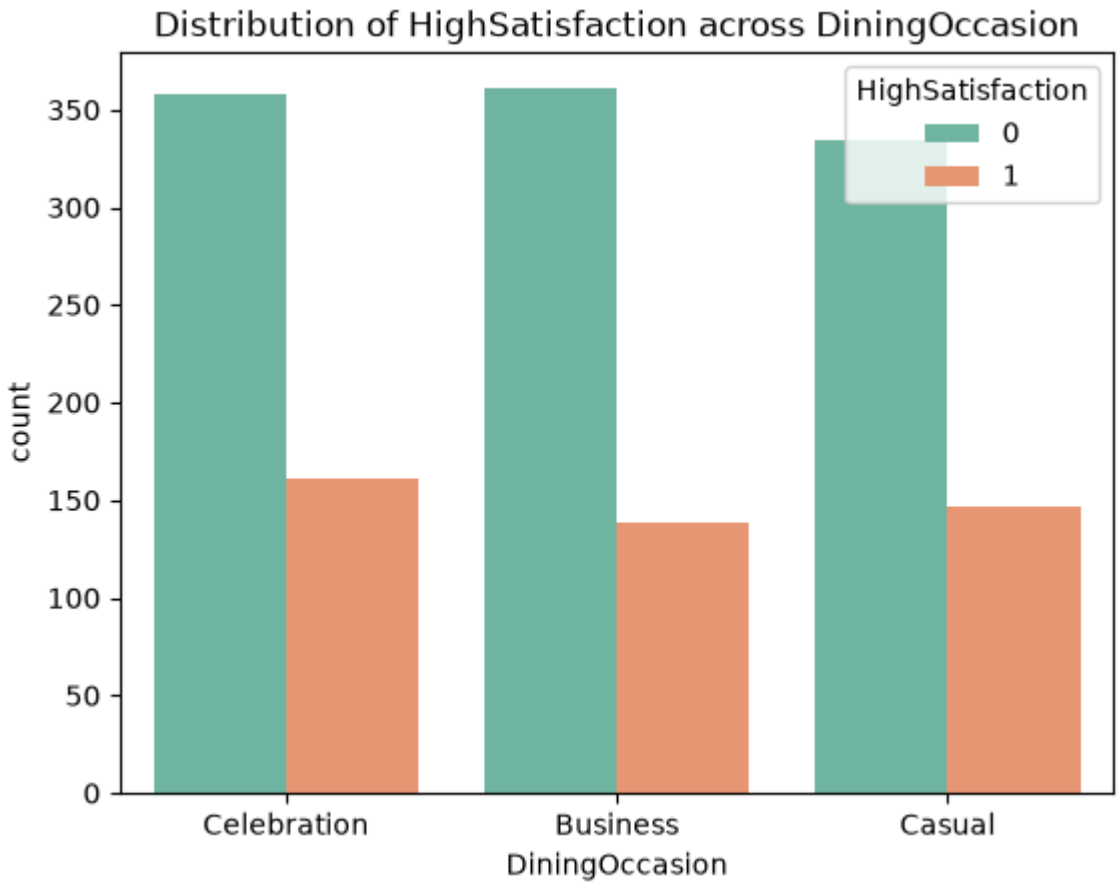
In [68]:
```
sns.countplot(data=df,x="Gender",palette="crest",order=df["Gender"].value_counts().index)
plt.show()
```



In [69]:
```
# Distribution of the target variable
plt.figure(figsize=(8, 4))
sns.countplot(x=df['HighSatisfaction'])
plt.title('Distribution of HighSatisfaction')
plt.show()
```



In [70]:
```
sns.countplot(x="DiningOccasion", hue='HighSatisfaction',palette="Set2",data=df)
plt.title(f'Distribution of HighSatisfaction across DiningOccasion')
plt.show()
```

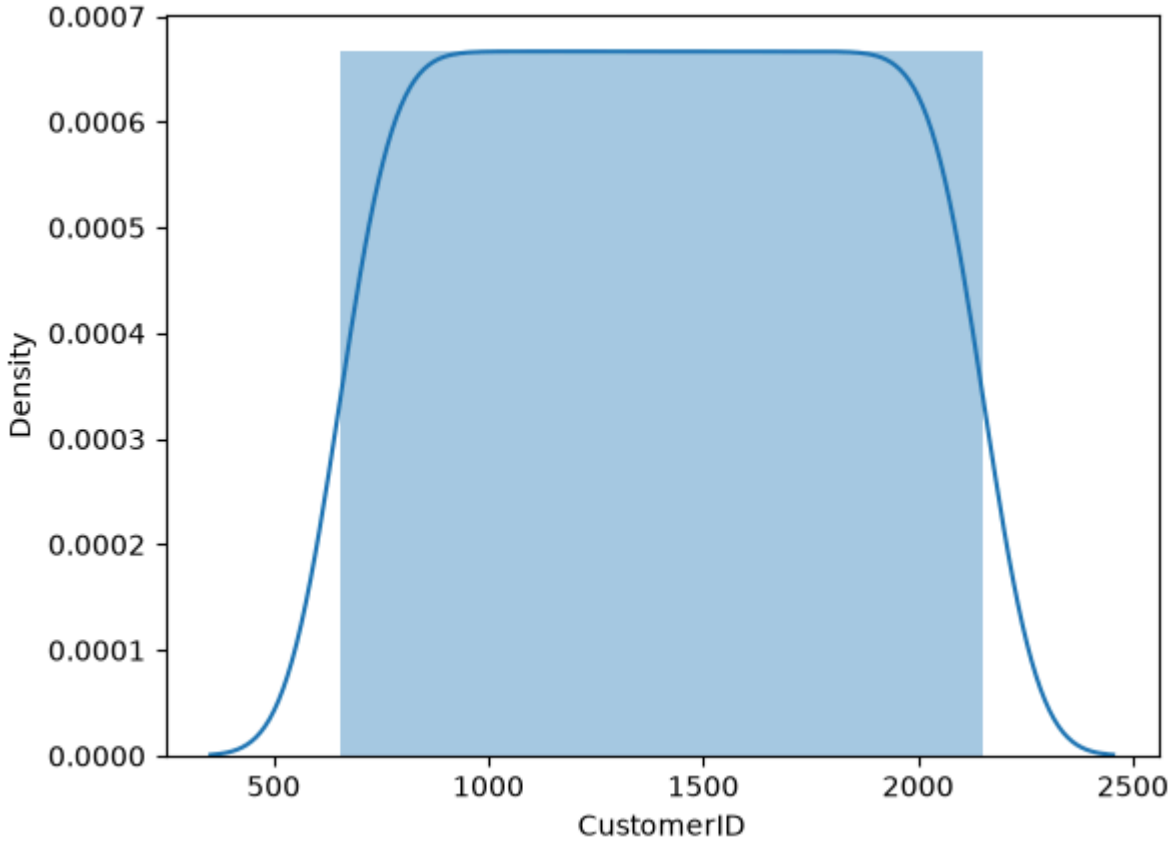


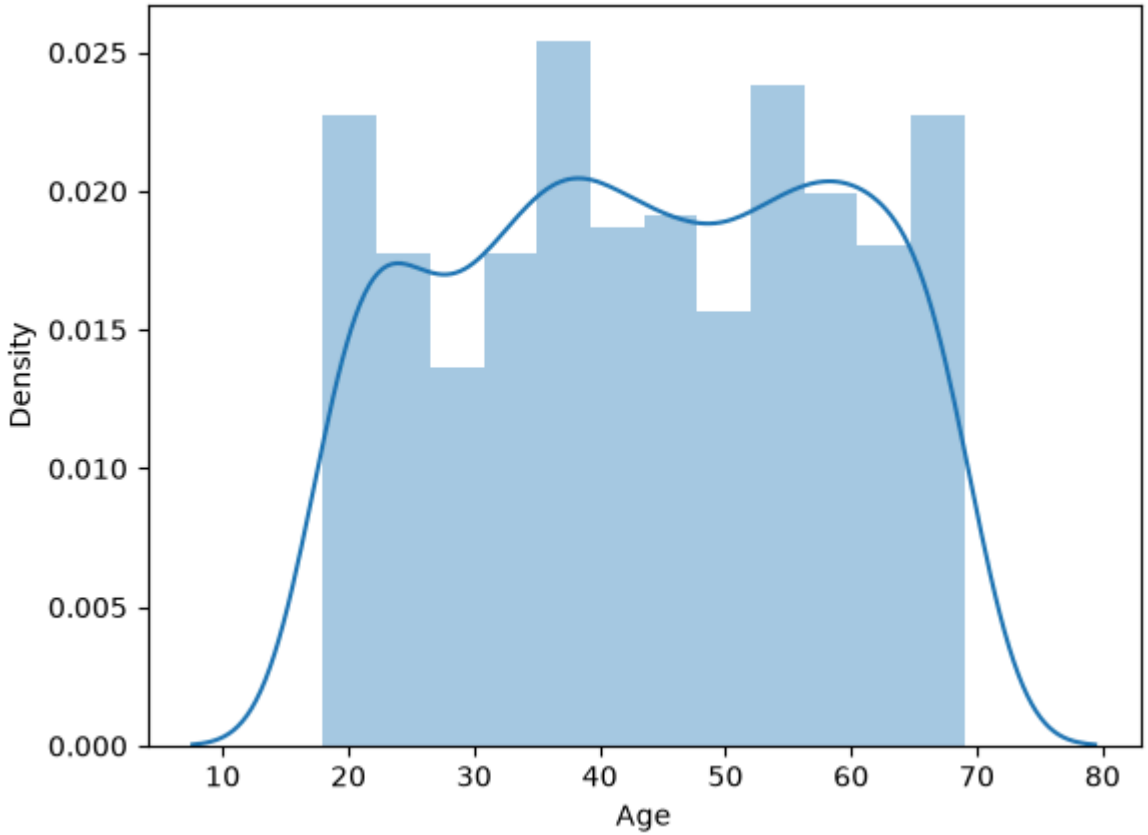
In [71]:
```
#SPLITTING

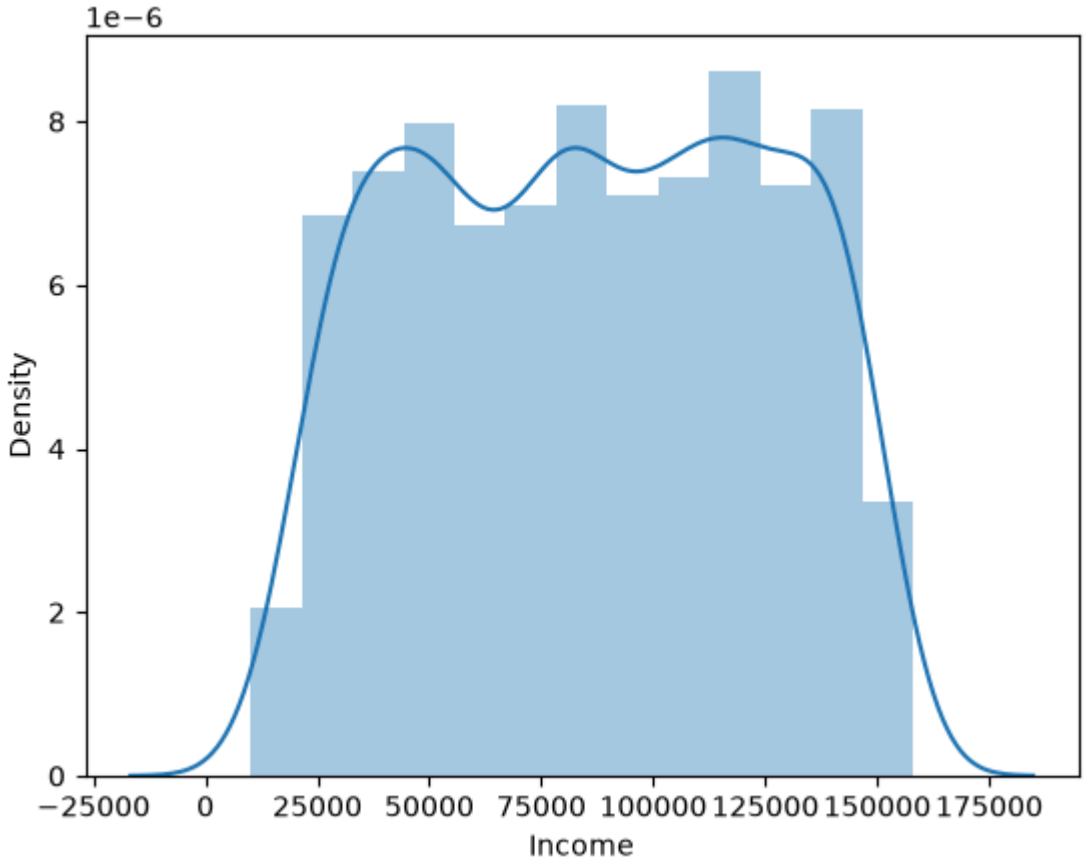
df_cat=df.select_dtypes(["object"])
df_num=df.select_dtypes(["int64","float64"])
```

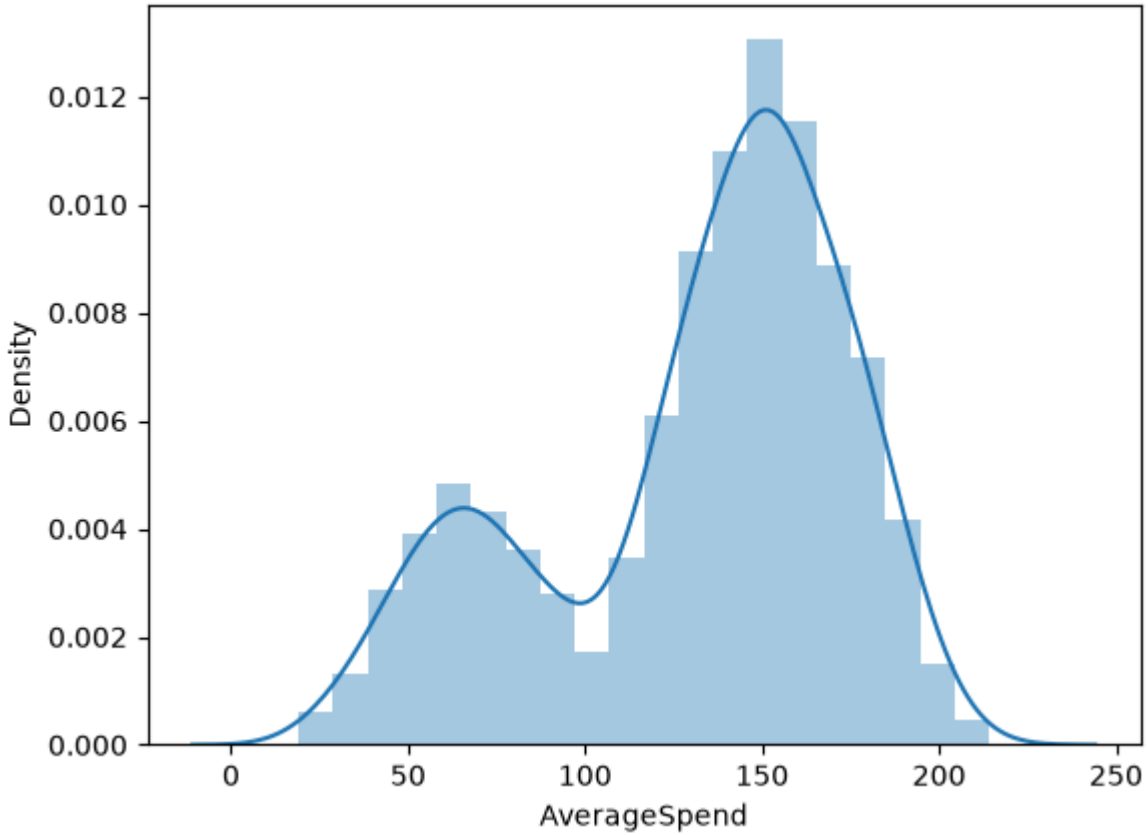
In [72]:
```
#Numerical Analysis ==== Checking Data Distribution of each through skewness

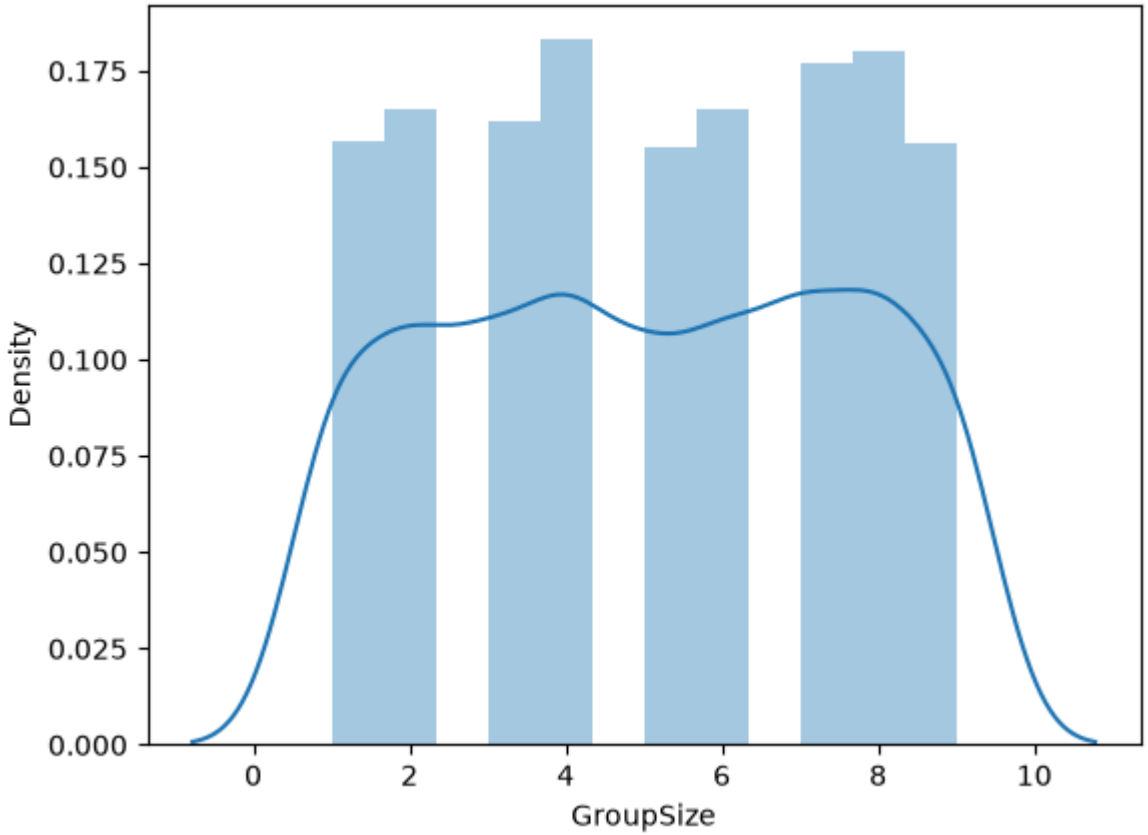
for column in df_num:
    sns.distplot(df_num[column])
    plt.show()
```

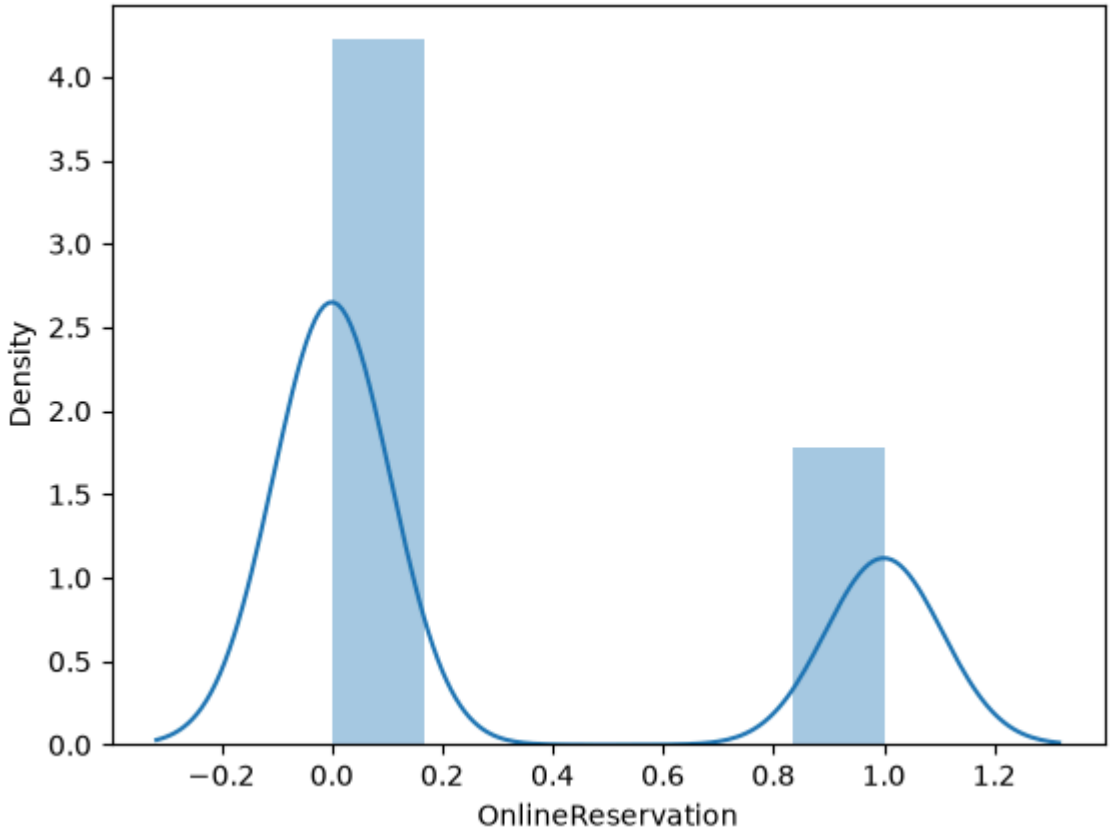

0.0007
0.0006
0.0005
0.0004
0.0003
0.0002
0.0001
0.0000
Density
500
1000
1500
2000
2500
CustomerID

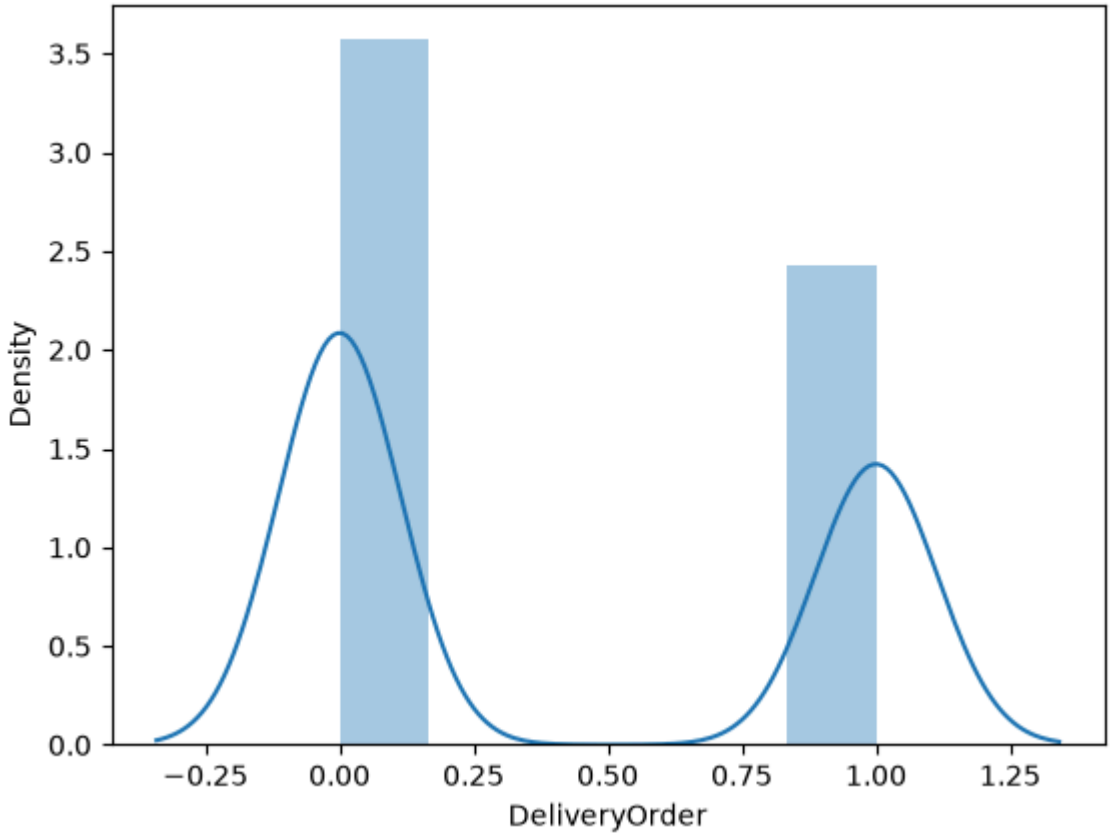

0.025
0.020
0.015
0.010
0.005
0.000
Density
10
20
30
40
50
60
70
80
Age

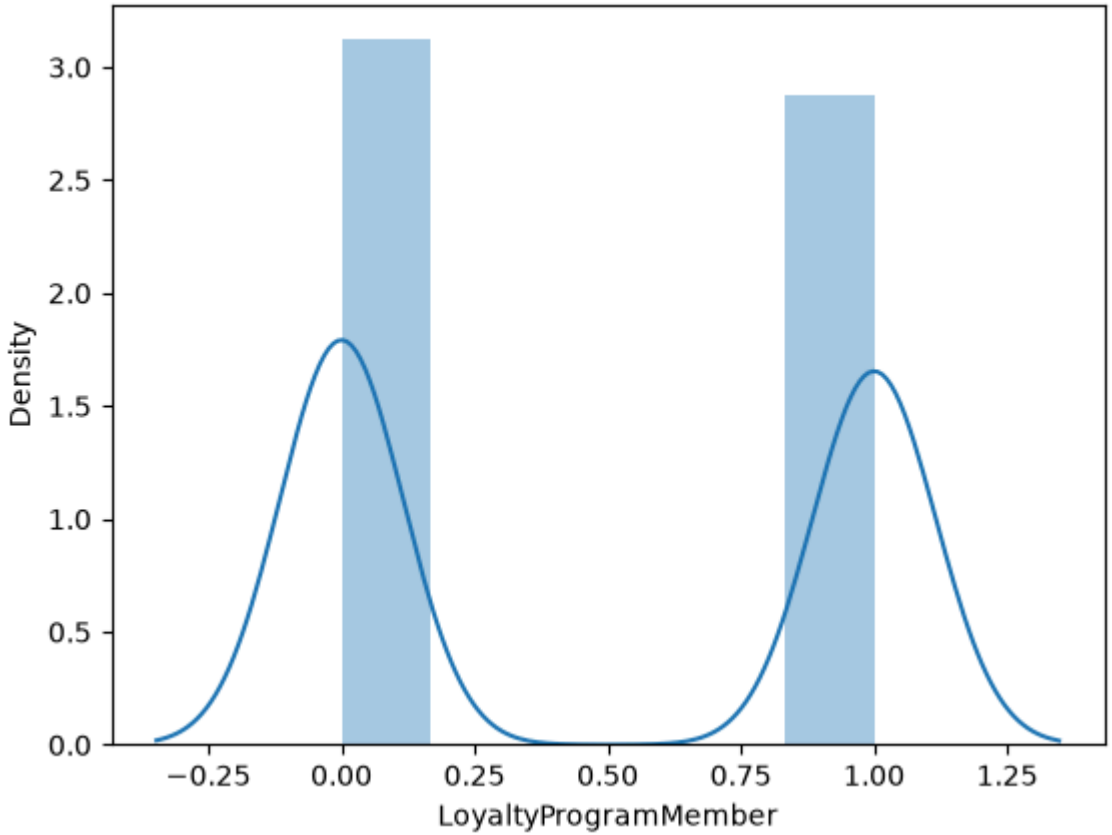

1e−6
8
6
4
2
0
Density
−25000
0
25000
50000
75000
100000
125000
150000
175000
Income

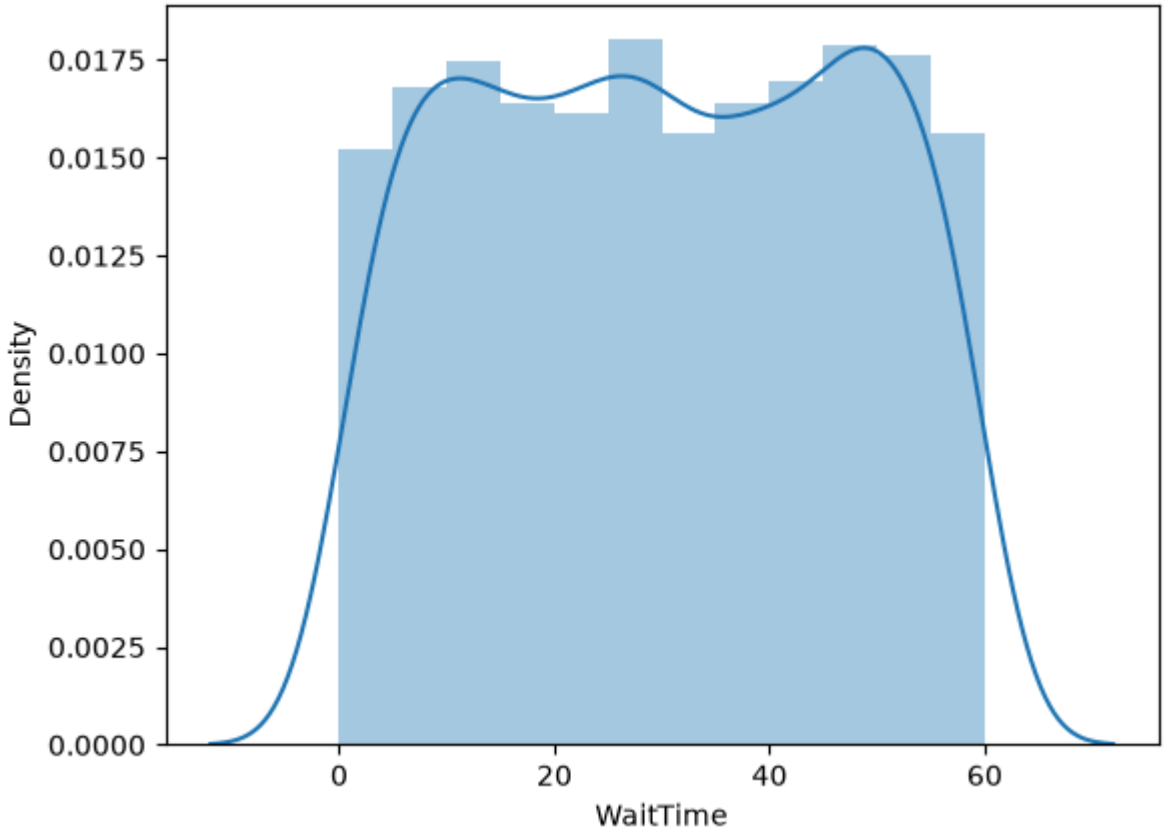

0.012
0.010
0.008
0.006
0.004
0.002
0.000
Density
0
50
100
150
200
250
AverageSpend

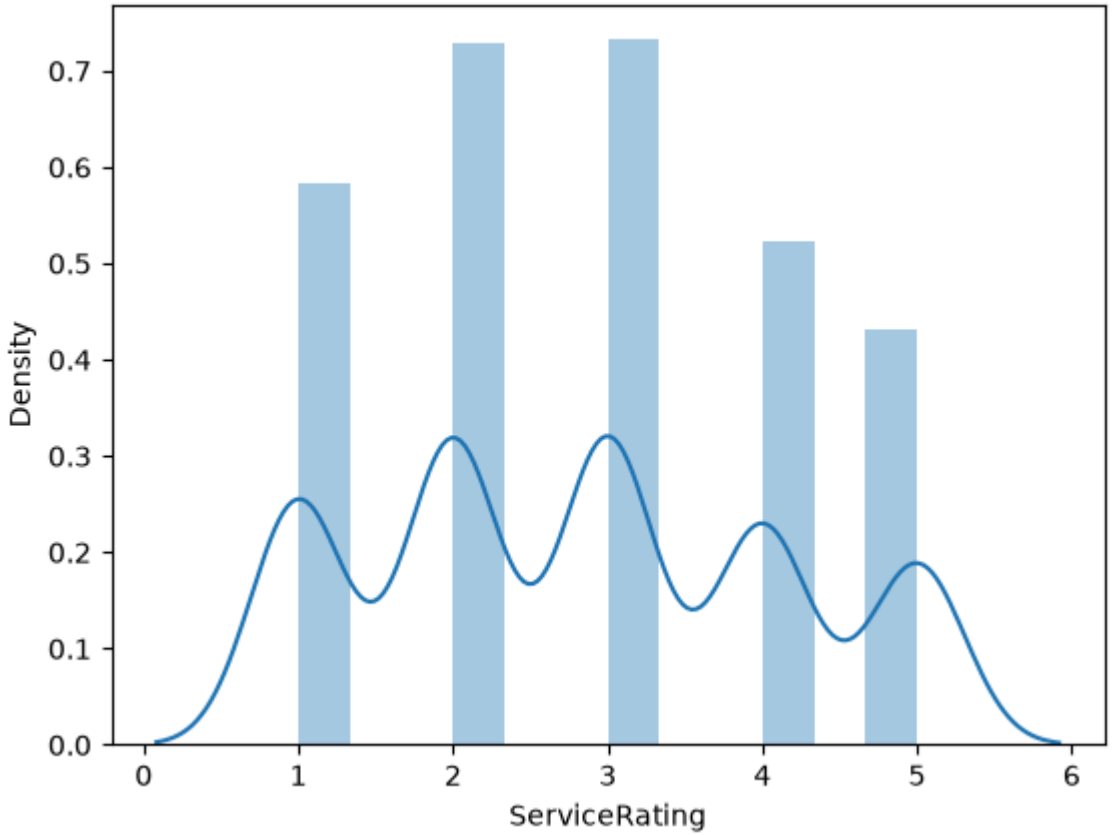

0.175
0.150
0.125
0.100
0.075
0.050
0.025
0.000
Density
0
2
4
6
8
10
GroupSize

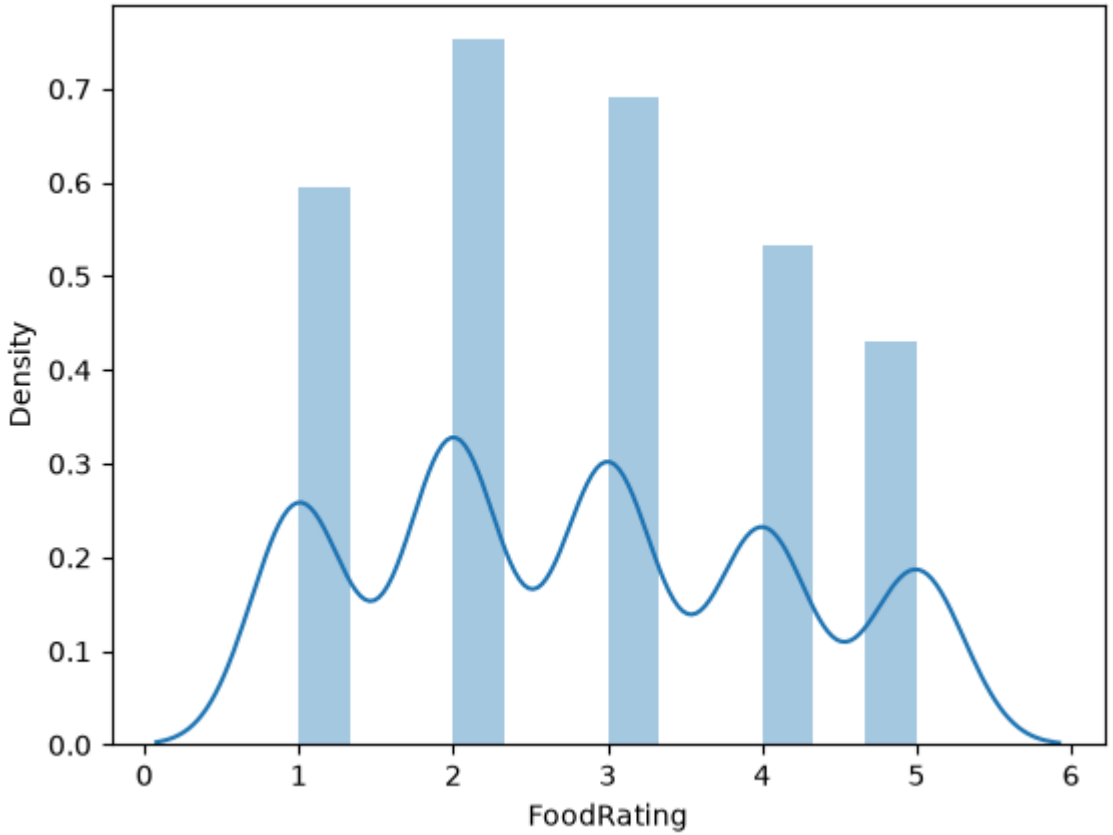

Density
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
−0.2
0.0
0.2
0.4
0.6
0.8
1.0
1.2
OnlineReservation

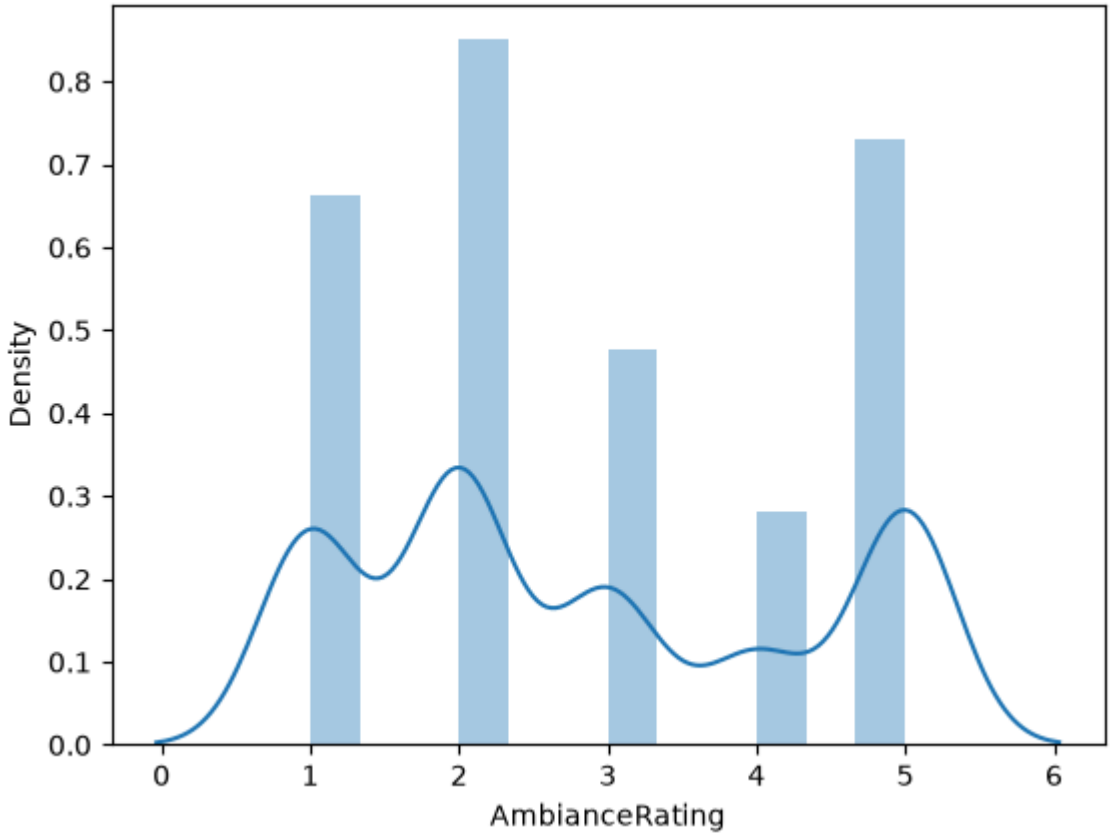

3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Density
−0.25
0.00
0.25
0.50
0.75
1.00
1.25
DeliveryOrder

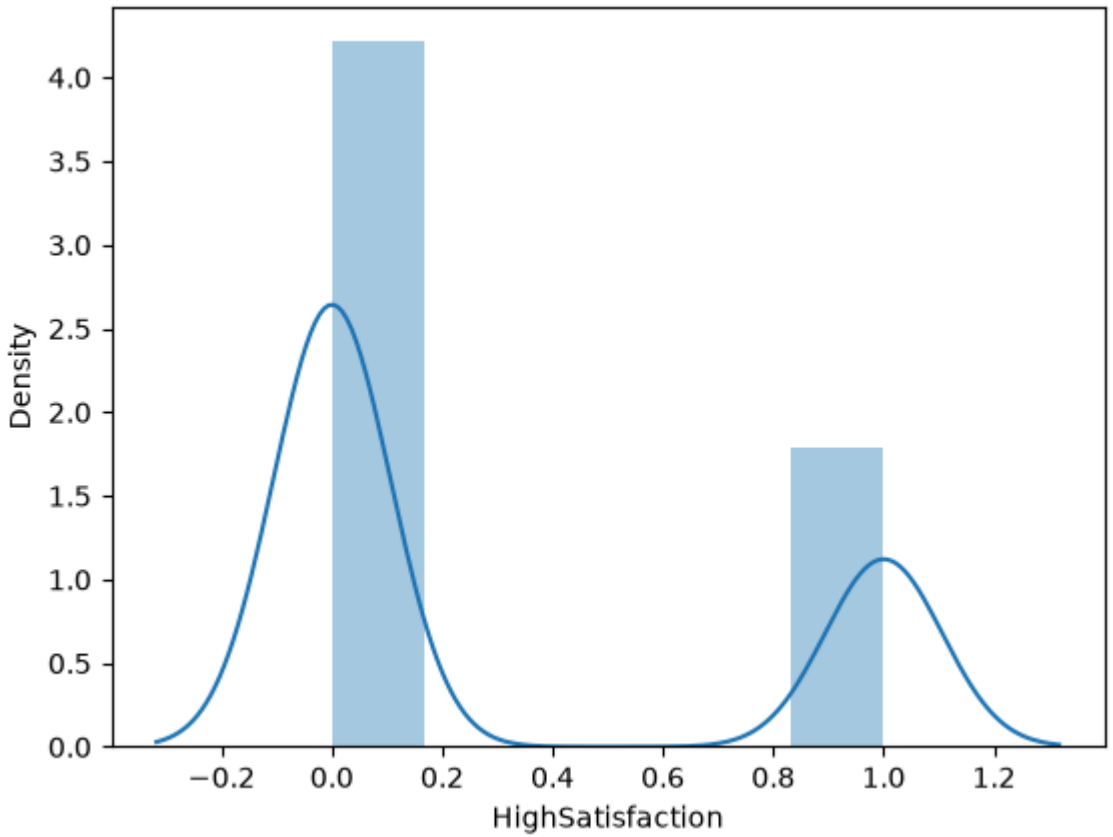

3.0
2.5
2.0
1.5
1.0
0.5
0.0
Density
−0.25
0.00
0.25
0.50
0.75
1.00
1.25
LoyaltyProgramMember


0.0175
0.0150
0.0125
0.0100
0.0075
0.0050
0.0025
0.0000
Density
0
20
40
60
WaitTime


0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.0
Density
0
1
2
3
4
5
6
ServiceRating


0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.0
Density
0
1
2
3
4
5
6
FoodRating


Density
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0
1
2
3
4
5
6
AmbianceRating



In [73]:
```
from scipy.stats import skew
```

In [74]:
```
for col in df_num:
    print("SKEWNESS of",col,":",skew(df_num[col]))
    print("**********************")
```

```
SKEWNESS of CustomerID : 0.0
**********
SKEWNESS of Age : -0.04711070131475571
**********
SKEWNESS of Income : -0.02933523825345021
**********
SKEWNESS of AverageSpend : -0.6788424392132338
**********
SKEWNESS of GroupSize : -0.023796746390097732
**********
SKEWNESS of OnlineReservation : 0.8902738144645338
**********
SKEWNESS of DeliveryOrder : 0.38564181948237874
**********
SKEWNESS of LoyaltyProgramMember : 0.0800640769025437
**********
SKEWNESS of WaitTime : -0.009509783294182736
**********
SKEWNESS of ServiceRating : 0.1812196636639055
**********
SKEWNESS of FoodRating : 0.1969687621917002
**********
SKEWNESS of AmbianceRating : 0.28890057087310045
**********
SKEWNESS of HighSatisfaction : 0.8832927728041264
**********
```

In [75]:
```
from sklearn.preprocessing import LabelEncoder
```

In [76]:
```
col=["CustomerID","Age","Gender","Income","VisitFrequency","AverageSpend","PreferredCuisine","TimeOfVisit","GroupSize","Dining

for i in col:
    le=LabelEncoder()
    df[i]=le.fit_transform(df[i])    #shortcut splitting
```

In [77]:
```
df
```

Out[77]:

	CustomerID	Age	Gender	Income	VisitFrequency	AverageSpend	PreferredCuisine	TimeOfVisit	GroupSize	DiningOccasion	Meal
0	64	15	1	1106	1	1368	4	0	7	2	
1	138	39	0	962	3	1321	4	2	2	2	
2	301	38	1	1192	2	1354	0	0	8	2	
3	311	40	0	1288	1	1334	3	2	0	0	
4	402	14	0	1074	3	1466	1	1	6	0	
...	...	...	...	...	...	...	...	...	...	...	
1495	1320	41	1	314	2	205	3	1	4	1	
1496	1359	4	0	326	3	160	0	1	7	2	
1497	1387	41	0	611	3	499	4	2	2	1	
1498	1432	13	1	541	3	571	1	1	1	2	
1499	1461	44	0	7	2	20	0	2	0	0	

1500 rows × 19 columns

In [78]:
```
#FEATURES TARGET

X=df.drop("HighSatisfaction",axis=1)
Y=df["HighSatisfaction"]
```

In [79]:
```
X
```

Out[79]:

	CustomerID	Age	Gender	Income	VisitFrequency	AverageSpend	PreferredCuisine	TimeOfVisit	GroupSize	DiningOccasion	Meal
0	64	15	1	1106	1	1368	4	0	7	2	
1	138	39	0	962	3	1321	4	2	2	2	
2	301	38	1	1192	2	1354	0	0	8	2	
3	311	40	0	1288	1	1334	3	2	0	0	
4	402	14	0	1074	3	1466	1	1	6	0	
...	...	...	...	...	...	...	...	...	...	...	
1495	1320	41	1	314	2	205	3	1	4	1	
1496	1359	4	0	326	3	160	0	1	7	2	
1497	1387	41	0	611	3	499	4	2	2	1	
1498	1432	13	1	541	3	571	1	1	1	2	
1499	1461	44	0	7	2	20	0	2	0	0	

1500 rows × 18 columns

In [80]:
```
Y
```

Out[80]:
```
0       0
1       0
2       0
3       0
4       0
       ..
1495    1
1496    1
1497    1
1498    1
1499    1
Name: HighSatisfaction, Length: 1500, dtype: int64
```

In [81]:
```
#TRAINING TESTING SPLITTING

from sklearn.model_selection import train_test_split
```

In [82]:
```
X_train,X_test,Y_train,Y_test=train_test_split(X,Y,test_size=0.3,random_state=1)
```

In [83]:
```
X_train
```

Out[83]:

	CustomerID	Age	Gender	Income	VisitFrequency	AverageSpend	PreferredCuisine	TimeOfVisit	GroupSize	DiningOccasion	Meal
1017	852	47	0	364	1	391	3	0	3	2	
188	102	44	1	1056	2	1310	4	1	0	1	
189	106	47	1	754	3	523	3	0	8	0	
1422	257	51	0	537	3	772	0	0	2	0	
1227	837	28	0	587	3	440	1	0	3	1	
...	...	...	...	...	...	...	...	...	...	...	
715	1290	18	1	95	2	192	4	0	0	0	
905	164	20	1	1233	3	1253	1	0	4	0	
1096	604	12	1	432	3	634	2	0	8	1	
235	466	34	1	960	1	959	2	0	7	2	
1061	451	40	0	1190	0	1481	2	0	4	1	

1050 rows × 18 columns

In [84]:
```
X_test
```

Out[84]:

	CustomerID	Age	Gender	Income	VisitFrequency	AverageSpend	PreferredCuisine	TimeOfVisit	GroupSize	DiningOccasion	Meal
91	282	46	0	197	1	58	4	0	0	2	
75	8	9	1	1452	3	975	2	0	4	2	
1264	1458	5	0	967	2	987	1	0	2	2	
330	1351	34	0	340	2	786	3	1	3	2	
1349	326	39	1	472	3	481	1	1	8	2	
...	...	...	...	...	...	...	...	...	...	...	
581	399	43	0	549	1	631	0	0	3	0	
154	1087	36	1	74	3	89	2	0	3	2	
643	813	1	1	1236	1	973	2	1	4	0	
930	674	51	0	840	3	500	3	0	8	0	
1466	1469	21	1	941	2	1172	3	0	3	0	

450 rows × 18 columns

Logistic Regression

In [85]:
```
from sklearn.linear_model import LogisticRegression
```

In [86]:
```
log=LogisticRegression()
```

In [87]:
```
log.fit(X_train,Y_train)
```

Out[87]:

LogisticRegression

▶ Parameters

▶ Fitted attributes

In [88]:
```
log.score(X_train,Y_train)
```

Out[88]: 0.9114285714285715

In [89]:
```
y_predict=log.predict(X_train)
```

In [90]:
```
log.score(X_test,Y_test)
```

Out[90]: 0.8888888888888888

In [91]:
```
y_predtest=log.predict(X_test)
```

In [92]:
```
from sklearn.metrics import classification_report,confusion_matrix
```

In [93]:
```
print(classification_report(Y_test,y_predtest))
```

```
              precision    recall  f1-score   support

           0       0.93      0.91      0.92       317
           1       0.79      0.84      0.82       133

    accuracy                           0.89       450
   macro avg       0.86      0.88      0.87       450
weighted avg       0.89      0.89      0.89       450
```

Adaptive Boosting

In [94]:
```
from sklearn.ensemble import AdaBoostClassifier
```

In [95]:
```
ada=AdaBoostClassifier(n_estimators=100)

#100 Decision stumps with max_depth=1 will be created
```

In [96]:
```
ada.fit(X_train,Y_train)
```

Out[96]:

AdaBoostClassifier

▶ Parameters

▶ Fitted attributes

In [97]:
```
ada.score(X_train,Y_train)
```

Out[97]:
```
0.979047619047619
```

In [98]:
```
y_predict=ada.predict(X_train)
```

In [99]:
```
y_predtest=ada.predict(X_test)
```

In [100…
```
ada.score(X_test,Y_test)
```

Out[100…
```
0.9666666666666667
```

In [101…
```
from sklearn.metrics import classification_report
```

In [102…
```
print(classification_report(Y_test,y_predtest))
```

	precision	recall	f1-score	support
0	0.97	0.98	0.98	317
1	0.95	0.93	0.94	133
accuracy			0.97	450
macro avg	0.96	0.96	0.96	450
weighted avg	0.97	0.97	0.97	450

Decision Tree

In [103…
```
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from sklearn.tree import DecisionTreeClassifier
```

In [104…
```
#train_test_split

X_train,X_test,Y_train,Y_test=train_test_split(X,Y,test_size=0.3,random_state=1)
```

In [105…
```
dt=DecisionTreeClassifier()
```

In [106…
```
dt.fit(X_train,Y_train)
```

Out[106…

▾ DecisionTreeClassifier

▸ Parameters

▸ Fitted attributes

In [107…
```
dt.get_depth()
```

Out[107… 10

In [108…
```
dt.get_n_leaves()
```

Out[108… np.int64(46)

In [109…
```
dt.score(X_train,Y_train)
```

Out[109… 1.0

In [110…
```
y_predict=dt.predict(X_train)
```

In [111…
```
y_predtest=dt.predict(X_test)
```

In [112…
```
y_predtest
```

Out[112…
```
array([0, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 1, 1,
       1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 1, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0,
       0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1,
       1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0,
       0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0,
       0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0,
       0, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0,
       1, 0, 1, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1,
       0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0, 1,
       0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0,
       0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 1, 1,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1,
       0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0,
       0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1,
       0, 1, 0, 0, 0, 0, 0, 0, 0, 1])
```

In [113…
```
print(classification_report(Y_test,y_predtest))
```

	precision	recall	f1-score	support
0	0.95	0.97	0.96	317
1	0.93	0.89	0.91	133
accuracy			0.95	450
macro avg	0.94	0.93	0.94	450
weighted avg	0.95	0.95	0.95	450

Conclusion-After evaluating various classification models, Adaptive Boosting (AdaBoost) achieved the highest accuracy of 88%. In contrast, the decision tree model underperformed. This indicates that AdaBoost is the most effective model for this classification task, providing superior accuracy compared to other models tested.